DAGs, Topological Sort & SCCs

Ordering dependencies, DP on DAGs, and collapsing cycles

Learning goals

- Define DAGs and **topological orders**; know when one exists
- Compute a topological order with **Kahn's algorithm** and with **DFS**
- Detect a **directed cycle**
- Do **DP on a DAG**: longest path, path counting, min/max cost
- See every DP from Weeks 14–15 as a DAG
- Find **strongly connected components** (Kosaraju, Tarjan) and build the **condensation**

Motivation: Course Schedule (CSES 1679)

- n courses, m requirements "a must be taken before b"
- Find an order to take all courses, or report IMPOSSIBLE

```
1 → 2          valid orders: 3 1 4 2 5,  4 3 5 1 2, …
3 → 1
4 → 5          add 2 → 3: cycle 1 → 2 → 3 → 1  → IMPOSSIBLE
```

Directed graph + "before" constraints = **topological sort**.

DAGs and topological order

- **DAG** = directed acyclic graph
- **Topological order**: a listing of vertices such that every edge u → v has u **before** v
- Exists ⇔ the graph has **no directed cycle**
  - cycle ⇒ impossible (someone on the cycle would have to precede itself)
  - DAG ⇒ some vertex has in-degree 0; put it first, remove it, repeat

Usually **many** valid orders; problems typically accept any.

Kahn's algorithm (BFS-style)

```
for (auto [a, b] : edges) { adj[a].push_back(b); indeg[b]++; }
queue<int> q;
for (int v = 1; v <= n; v++) if (indeg[v] == 0) q.push(v);
vector<int> order;
while (!q.empty()) {
    int u = q.front(); q.pop();
    order.push_back(u);
    for (int v : adj[u])
        if (--indeg[v] == 0) q.push(v);
}
if (order.size() < n) → cycle!
```

O(n + m). Want the lexicographically smallest order? Use a **min-heap** instead of a queue.

Kahn trace (CSES 1679 sample)

Edges: 1→2, 3→1, 4→5. Initial in-degrees: 1:1, 2:1, 3:0, 4:0, 5:1

step	queue	pop	order
0	3 4		
1	4 1	3	3
2	1 5	4	3 4
3	5 2	1	3 4 1
4	2	5	3 4 1 5
5		2	3 4 1 5 2 ✓

DFS-based topological sort

Run DFS; when a vertex **finishes** (all descendants done), append it. Reverse the list.

```
void dfs(int u) {
    state[u] = 1;                          // on the current path (gray)
    for (int v : adj[u]) {
        if (state[v] == 1) hasCycle = true;   // back edge!
        else if (state[v] == 0) dfs(v);
    }
    state[u] = 2;                          // finished (black)
    order.push_back(u);
}
// reverse(order) is a topological order
```

Why: for an edge u → v, v always finishes before u (if no cycle).

Cycle detection with colours

- **white (0)**: not visited · **gray (1)**: on the recursion stack · **black (2)**: done
- Edge into a **gray** vertex = back edge = **directed cycle**
- Edge into a black vertex is fine (cross / forward edge)

⚠ Undirected cycle detection is different ("visited and not my parent").
⚠ A plain visited array **cannot** detect directed cycles.

To print the cycle (CSES 1678 Round Trip II): keep parents, walk back from u to v.

DP on a DAG

Process vertices in **topological order** — every predecessor is final before you use it.

Longest path from 1 to n (CSES 1680 Longest Flight Route):

```
best[1] = 1;  // others = −∞
for (int u : order) {
    if (best[u] == NEG) continue;          // not reachable from 1
    for (int v : adj[u])
        if (best[u] + 1 > best[v]) { best[v] = best[u] + 1; par[v] = u; }
}
```

O(n + m) — longest path is NP-hard in general graphs, **easy on DAGs**.

More DAG DPs

Question	Transition along u → v
longest / shortest path (any weights!)	`best[v] = max/min(best[v], best[u] + w)`
number of paths 1 → n (CSES 1681 Game Routes)	`ways[v] += ways[u]` (mod p)
reachable from source?	`reach[v]
min/max number of edges	as longest/shortest with w = 1

Negative weights are fine on a DAG — no Bellman-Ford needed.

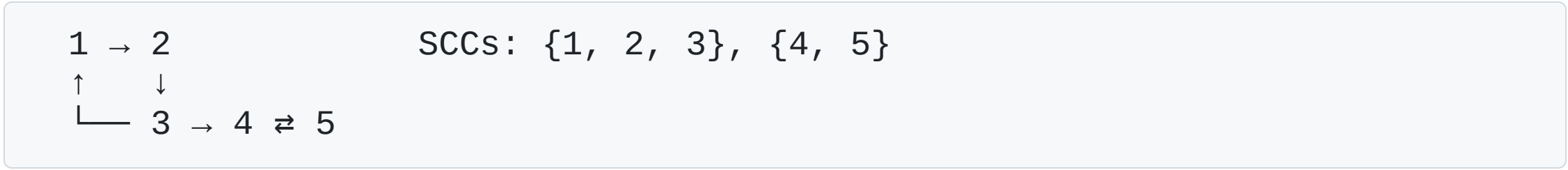
Every DP is a DAG

- States = vertices, "depends on" = edges, base cases = sources
- Week 15 grid paths: cells, edges right/down, row-major = topological order
- Knapsack: (item, capacity) states, edges to (item + 1, ...)
- Memoised recursion = **DFS-based topological sort** done implicitly

If your DP state graph has a **cycle**, it's not a DP — it's a shortest-path problem (Dijkstra / BFS).

When the graph has cycles: SCCs

Strongly connected component: maximal set where every vertex reaches every other.



- Every vertex is in exactly one SCC
- Contract each SCC to one node → **condensation graph**, always a **DAG**
- Then: DAG techniques apply!

Kosaraju's algorithm

1. DFS on G; record vertices in **finishing order**
2. Build the **reversed** graph G^R
3. Process vertices in **decreasing finishing time**; each DFS on G^R from an unassigned vertex collects exactly one SCC

Why: the vertex finishing last lies in a **source** SCC of the condensation; in G^R that SCC is a sink, so the DFS can't leak out.

Two DFS passes → **O(n + m)**. Easy to write iteratively.

Kosaraju: pass 1 (iterative finishing order)

```
vector<size_t> it(n + 1, 0);                // next neighbour to try
for (int s = 1; s <= n; s++) {
    if (vis[s]) continue;
    vector<int> st = {s}; vis[s] = true;
    while (!st.empty()) {
        int u = st.back();
        if (it[u] < g[u].size()) {
            int v = g[u][it[u]++];
            if (!vis[v]) { vis[v] = true; st.push_back(v); }
        } else { finish.push_back(u); st.pop_back(); }
    }
}
```

Pass 2: loop `finish` backwards, flood-fill on `rg`, label `comp[]`.

Tarjan's algorithm (one pass)

`tin[u]` = discovery time, `low[u]` = smallest `tin` reachable via the DFS subtree + one edge into the **current stack**.

```
void tarjan(int u) {
    tin[u] = low[u] = ++timer; stk.push_back(u); onStack[u] = true;
    for (int v : g[u]) {
        if (!tin[v]) { tarjan(v); low[u] = min(low[u], low[v]); }
        else if (onStack[v]) low[u] = min(low[u], tin[v]);
    }
    if (low[u] == tin[u]) {                  // u is the SCC root
        int v;
        do { v = stk.back(); stk.pop_back(); onStack[v] = false; comp[v] = nComp; }
        while (v != u);
        nComp++;
    }
}
```

Tarjan details

- SCCs are produced in **reverse topological order** of the condensation
  → for DP on the condensation, process ids 0, 1, 2, … and successors are already done
- `onStack` check is essential: edges to finished SCCs must be ignored
- Recursive: depth up to n. Fine on CSES/Codeforces; Kosaraju iterative is the safe choice elsewhere
- Same `tin` / `low` idea finds **bridges and articulation points** (National track)

Condensation + DP: Coin Collector (CSES 1686)

Rooms with coins, one-way tunnels; start anywhere, collect max coins (revisits allowed).

1. Inside an SCC you can collect **everything** → `val[c]` = sum of its coins
2. Condensation is a DAG → longest **vertex-weighted** path

```
for (int c = 0; c < nComp; c++) {          // Tarjan order: successors first
    best[c] = val[c];
    for (int d : dag[c]) best[c] = max(best[c], val[c] + best[d]);
    ans = max(ans, best[c]);
}
```

Sample: SCC {1, 2} worth 9, then room 4 worth 7 → **16**.

Other SCC applications

- **Is the graph strongly connected?** (CSES 1682 Flight Routes Check) — exactly one SCC; otherwise print a pair (a, b) with no a → b route
- **Minimum edges to add** to make it strongly connected: max(#sources, #sinks) of the condensation (if > 1 SCC)
- **2-SAT** (National track): x and ¬x in the same SCC ⇒ unsatisfiable
- Reachability questions on general digraphs → condense first

Complexity summary

Algorithm	Time	Memory
Kahn / DFS topological sort	O(n + m)	O(n + m)
Directed cycle detection	O(n + m)	O(n)
DP on DAG	O(n + m)	O(n)
Kosaraju (2 DFS + reversed graph)	O(n + m)	O(n + m)
Tarjan (1 DFS)	O(n + m)	O(n) extra
Condensation	O(n + m)	O(n + m)

Common pitfalls

- Forgetting to check `order.size() == n` → printing a partial order for a cyclic graph
- Detecting directed cycles with a visited array only (needs gray/black)
- DAG DP that starts from all vertices when the problem says "from city 1"
- Using `0` instead of $-\infty$ for unreachable states in longest path
- Tarjan without the `onStack` test → merges unrelated SCCs
- Duplicate edges in the condensation (harmless for max-DP, **wrong for counting**)
- Recursion depth with recursive DFS/Tarjan on 10^5-long chains

Practice problems

- CSES 1679 Course Schedule · CSES 1678 Round Trip II
- CSES 1680 Longest Flight Route · CSES 1681 Game Routes
- AtCoder DP Contest G Longest Path
- CSES 1682 Flight Routes Check · CSES 1683 Planets and Kingdoms
- CSES 1686 Coin Collector · CSES 1757 Course Schedule II
- CF 510C Fox And Names · CF 427C Checkposts

Summary

- Topological order exists ⇔ DAG; find it with **Kahn** (in-degrees) or **DFS** (reverse finish order)
- Directed cycle = edge into a gray vertex, or Kahn leaves vertices behind
- DP on a DAG in topological order: longest paths, counting paths — O(n + m)
- Every DP is a DAG; cycles in the state graph mean "use shortest paths"
- **SCCs** collapse cycles; the condensation is a DAG → reuse DAG DP
- Next week: LCA, binary lifting and Euler tours on trees